Interview Prep Guide

Credit Card Fraud Detection Using Machine Learning

Written in simple English. Every sample answer is in first person, so you can say it out loud. All numbers come from your own notebook and README.

Read this first (2 minutes)

What I found in your repo. Your zip has two files: a README and one notebook called *Logistic Regression*. So the notebook proves one model. Everything below about results is built on that notebook.

Three things to be careful about in the interview

- **The README says KNN and Decision Tree scored 100.** That is not in your notebook. A perfect score on fraud data is a warning sign for most interviewers. Only say it if you have that code and can explain it. Otherwise say the honest line in Question 19.
- **You only reported accuracy.** On fraud data, accuracy alone can fool you. Be ready to talk about precision and recall (explained in Section 5). Do not say you measured them. You did not.
- **Your README says "We".** If asked whether it was a team project, answer with exactly what *you* did.

The role, company and round were not filled in, so I wrote this for a general data or fraud analytics interview. If you tell me the job title, I can add a page that links this project to that job.

What is inside

1	30-second answer
2	2-minute answer, step by step
3	A simple real-life story (analogy)
4	Project facts at a glance, and why each tool
5	Key ideas explained like you are teaching a beginner
6	20 likely questions with easy answers
7	Tough follow-up questions
8	Challenges and what you learned
9	Tips for speaking clearly
10	One-page cheat sheet for tomorrow morning

1. Your 30-second answer

Use this when they say: *"Tell me about your project."* Say it slowly. It should take about 30 seconds.

> "I built a machine learning project that finds fraud in credit card transactions. I used a public Kaggle dataset with about 285,000 transactions. Only 492 of them were fraud, which is a very small part. To fix this imbalance, I took an equal number of normal and fraud transactions. Then I trained a Logistic Regression model in Python. It reached about 92 percent accuracy on test data. From this project I learned how to work with imbalanced data and why accuracy alone is not enough."

Memory trick: Problem, Data, Fix, Model, Result, Learning.

2. Your 2-minute answer

Use this when they say: *"Explain it in detail."* Five steps. Pause a little between steps.

Step 1: The problem

> "Credit card fraud costs people and banks a lot of money. A bank has millions of transactions, and no person can check them all. So I wanted a model that can look at a transaction and say: fraud or normal."

Step 2: The data

> "I used the Kaggle credit card dataset. It has 284,807 transactions from European cardholders in 2013. Each row has 30 inputs and one answer called Class. Class is 1 for fraud and 0 for normal. 28 inputs are named V1 to V28. They are hidden numbers, because the bank wanted to protect customer privacy. The other two are Time and Amount. There were no missing values."

Step 3: The big challenge - imbalance

> "Only 492 transactions were fraud. That is about 0.17 percent. If a model just says 'everything is normal', it is right 99.8 percent of the time but catches no fraud. So I made a balanced sample: all 492 fraud transactions and 492 random normal ones. That gave me 984 rows."

Step 4: The model

> "I split the data into 80 percent for training and 20 percent for testing. That is 787 rows to train and 197 to test. I used stratify, so both parts have the same fraud share. Then I trained a Logistic Regression model with scikit-learn."

Step 5: The result and learning

> "The model got 93.5 percent accuracy on training data and 91.9 percent on test data. These two numbers are close, so the model is not just memorising. I also learned accuracy is not enough for fraud. Next time I would also check precision and recall, and test on real, unbalanced data."

3. A simple real-life story (analogy)

Use this if the interviewer is not technical, or if you want to explain the idea in 20 seconds.

> "Imagine a huge basket with 285,000 apples. Only 492 are rotten. An inspector who looks at this basket will learn very little about rotten apples, because almost every apple he sees is good. So I made a smaller training basket: 492 rotten apples and 492 good ones. Now the inspector sees both kinds equally and learns what rotten looks like. Logistic Regression is the inspector. For each apple, it gives a score from 0 to 1 for 'chance of being rotten'. If the score is above 0.5, it says rotten."

The catch (say this to sound mature): in real life the inspector will see a basket that is 99.8 percent good apples again. So I should also test him on a normal, unbalanced basket before trusting him.

4. Project facts at a glance

Item	What it is (from your files)
Data source	Kaggle credit card fraud dataset (European cardholders, 2013, over two days)
Size	284,807 rows, 31 columns (Time, V1-V28, Amount, Class)
Fraud vs normal	492 fraud (about 0.17%) and 284,315 normal
Missing values	None
Average amount	Normal: 88.29. Fraud: 122.21. Middle value (median): normal 22.00, fraud 9.25
Balancing method	Under-sampling: 492 random normal + all 492 fraud = 984 rows
Train / test split	80 / 20, stratified. 787 train rows, 197 test rows. random_state=2
Model in notebook	Logistic Regression (scikit-learn)
Training accuracy	93.52%
Test accuracy	91.88%
Libraries	numpy, pandas, scikit-learn

Fun fact for the interview: average fraud rows are very different from normal rows in your table. For example, the average of V3 is about -7.03 for fraud and about 0.01 for normal. This is the clue the model uses to tell them apart.

Why each tool, and what else you could have used

Tool / choice	Why I used it (say this)	Other options
Python	Simple to read and has strong data libraries.	R, SQL for data prep
pandas	Loads the CSV and lets me filter, count and compare easily.	Polars, Excel
scikit-learn	Ready-made, trusted tools for splitting data and training models.	TensorFlow, XGBoost library
Logistic Regression	Simple and fast, and gives a fraud probability. Good first model, easy to explain.	Random Forest, XGBoost, KNN, SVM, Decision Tree
Under-sampling	Quick way to give the model equal fraud and normal examples.	SMOTE (make extra fraud samples), class weights, over-sampling

Tool / choice	Why I used it (say this)	Other options
Accuracy score	Easy first check on how often predictions are right.	Precision, recall, F1, ROC-AUC

5. Key ideas, explained like you are teaching a beginner

Word	Plain meaning
Machine learning	Teaching a computer by showing it many examples, so it can make a guess on new ones. Here: many past transactions marked fraud or normal.
Features and target	Features are the clues (Time, Amount, V1 to V28). The target is the answer we want to guess (Class: fraud or not). In your code: X is the clues, Y is the answer.
PCA (why V1 to V28)	PCA is a way to mix the original columns into new columns. The bank used it to hide private details like merchant and card number. So we cannot say what V1 means. We only know it still carries useful signal.
Imbalanced data	One group is much bigger than the other. Here normal is 99.83 percent and fraud is 0.17 percent. The model can get lazy and just guess 'normal'.
Under-sampling	Throw away many rows from the big group so both groups are equal. Good: simple. Bad: you lose information from the thrown-away rows.
Train / test split	Train data is the practice book. Test data is the exam with new questions. We keep them apart so we see real learning, not memorising.
Stratify	When splitting, keep the same fraud share in both train and test. Without it, the test set might get almost no fraud by bad luck.
random_state	A fixed number so the split is the same every time you run the code. Makes results repeatable.
Logistic Regression	Despite the name, it is a classification model. It gives each clue a weight, adds them up, and turns the total into a chance between 0 and 1. Above 0.5 means fraud.
Accuracy	Out of all predictions, how many were right. Your test: about 92 out of 100 were right.
Overfitting	The model memorises the practice book but fails the exam. You check it by comparing train and test scores. Yours are close (93.5 vs 91.9), so it looks fine.
Precision	Of all transactions the model called fraud, how many were really fraud. High precision means few false alarms.
Recall	Of all real fraud, how many did the model catch. High recall means few missed frauds.
Confusion matrix	A small 2 by 2 table that counts: fraud caught, fraud missed, normal wrongly flagged, normal correctly passed. Precision and recall come from it.

6. 20 likely questions with easy answers

Say these in your own words. Short sentences are better than perfect sentences.

1. Tell me about your project.

I built a model that detects credit card fraud. I used a Kaggle dataset of about 285,000 transactions, balanced it, trained Logistic Regression in Python, and got about 92 percent test accuracy.

2. Why did you choose this project?

Fraud is a real and growing problem, and it is a good example of messy, unbalanced data. It also let me practise the full flow: load data, explore, balance, train and test.

3. Where did the data come from, and what is in it?

It is from Kaggle. It has 284,807 transactions from European cardholders, with 30 input columns and a Class column. Class is 1 for fraud and 0 for normal.

4. What are V1 to V28?

They are hidden columns made by PCA, which mixes the original columns into new ones. The bank did this to protect privacy. I cannot say what each one means, but they still help the model.

5. How imbalanced is the data, and why is that a problem?

Only 492 of 284,807 are fraud, about 0.17 percent. A model could say 'normal' every time and look 99.8 percent accurate, while catching zero fraud. So I could not trust plain accuracy on the raw data.

6. How did you handle the imbalance?

I used under-sampling. I kept all 492 fraud rows and picked 492 normal rows at random. That gave me a balanced set of 984 rows.

7. What is the downside of under-sampling?

I throw away most of the normal data, so I lose information. Also the balanced test set does not look like real life, where fraud is rare. Next time I would try SMOTE or class weights.

8. Why Logistic Regression?

It is simple, fast, and gives a probability of fraud. It is a good first model and easy to explain. I wanted a baseline before trying harder models.

9. How does Logistic Regression work, in simple words?

It gives each input a weight, adds everything up, and turns the total into a number between 0 and 1. That number is the chance of fraud. If it is above 0.5, it says fraud.

10. How did you split the data? What is stratify?

I used 80 percent for training and 20 percent for testing. That is 787 and 197 rows. Stratify keeps the same share of fraud and normal in both parts.

11. Why did you set random_state?

So the split is the same each time, and my results can be repeated. One honest point: my normal-transaction sample did not have a fixed seed, so results may change a little between runs.

12. What results did you get?

93.5 percent accuracy on training data and 91.9 percent on test data.

13. Is your model overfitting?

It does not look like it. Train and test accuracy are close, about 1.6 points apart. Overfitting would show a big gap, like 99 percent on train and 80 on test.

14. Did you clean the data?

I checked for missing values and found none. The V columns were already processed by PCA. I did not scale Time and Amount, and that is something I would add.

15. What did you find in your data exploration?

Fraud rows look very different from normal rows in many V columns. For example, the average V3 is about -7 for fraud and about 0 for normal. Also, half of the fraud transactions were under 9.25, while the middle normal amount was 22.

16. Is accuracy the right measure for fraud?

Not by itself. In fraud, missing a real fraud and wrongly blocking a good customer both cost money. So I should also look at precision and recall. In this project I reported accuracy only, and I know that is a limit.

17. What is the difference between precision and recall?

Precision asks: when I say fraud, how often am I right. Recall asks: of all the real fraud, how much did I catch. For fraud, recall is often very important, but too many false alarms annoy customers, so we balance both.

18. What is a confusion matrix? Did you use one?

It is a table that counts correct and wrong predictions for each class. I did not include it in this notebook, but it is the first thing I would add, because precision and recall come from it.

19. Your README lists KNN, SVM and Decision Tree too. How did they do?

If you have the code and can explain it: Give the numbers you can show, and say what the model does in one line. **If you do not:** "The notebook in my repo is the Logistic Regression one, and that is the one I can walk you through fully. My README also lists KNN, SVM and Decision Tree as models to compare. I would not trust a perfect 100 percent score without checking for mistakes, so I would re-run them with proper precision and recall before quoting any numbers." This is honest and shows good judgement.

20. How would this work in a real bank?

> The model would score every new transaction. High scores would be blocked or sent to a human for review. It would need to be retrained often, because fraudsters change their tricks, and it would need to be tested at the real fraud rate before going live.

7. Tough follow-up questions

Pattern for tough questions: **agree honestly, then show how you would fix it.** Never argue and never bluff.

What would you improve?

> Four things. First, keep the test set natural, with real fraud rates. Second, report precision, recall, F1 and a confusion matrix. Third, try SMOTE or class weights instead of throwing data away. Fourth, compare more models such as Random Forest and XGBoost.

What went wrong or what are the limits?

> I only reported accuracy, and I tested on a balanced sample, so 91.9 percent will not hold on real data. I also used one model in the notebook. I see these as the next steps, not as surprises.

Your test set is balanced. Is that fair?

> Not fully. It shows how well the model separates the two groups when they are equal. In real life fraud is about 0.17 percent, so there will be far more false alarms. I would test on a set that keeps the real ratio.

Would you trust 92 percent accuracy in production?

> No, not on this evidence alone. I need to know how many frauds it catches and how many good customers it blocks. I would also watch it after launch and retrain it.

Is the 100 percent in your README believable?

> I would be careful with it. A perfect score in fraud usually means something is leaking or the test was too easy. I would re-check those models with precision, recall and a clean test set before I rely on that number.

How do you know it was not luck from the random sample?

> Good point. I did not fix a seed for the normal sample, so a different sample could give a slightly different score. I would repeat the run many times or use cross-validation and report the average.

Why not Random Forest or XGBoost?

> I started with a simple baseline so I could understand the problem first. Tree models usually do better on fraud data, so they are my next step.

Fraudsters change tricks. What then?

> Then the model gets old. This is called drift. I would watch performance every month, collect new fraud cases and retrain.

Which is worse: missing a fraud or blocking a good customer?

> Both cost money. A missed fraud means a direct loss. A false block hurts trust. The bank must pick the balance with its business team. I would show them how the numbers change at different score cut-offs.

Was this a team project? What did YOU do?

> Say only what is true. Example: "I did the data exploration, balancing, splitting, training and evaluation in the notebook, and I wrote the README." Change it to match reality.

8. Challenges and what you learned

You did not list your own challenges, so these come from your notebook. If your real story is different, use your real story.

Challenge 1: Very few fraud cases

> "The biggest problem was imbalance. Only 0.17 percent of rows were fraud. I solved it by balancing the data with under-sampling, so the model could learn both groups properly. I learned that the way you prepare data matters as much as the model."

Challenge 2: Hidden columns

> "V1 to V28 are hidden by PCA, so I could not explain them in business words. I compared the average value of each column for fraud and normal rows to see where they differ. I learned that you can still get useful clues from data you cannot fully read."

Challenge 3: Accuracy can mislead

> "At first accuracy sounds great. Then I understood that on fraud data it can hide missed fraud. Now I always ask: what kind of mistakes is the model making? That is why precision and recall are my next step."

What you learned (say as a closing line)

- How to explore a large dataset and spot differences between groups.
- Why imbalanced data needs special care, and how under-sampling works.
- How to split data properly and compare train and test scores.
- That one number, accuracy, is never the full story.

How this connects to a data or fraud analytics job

> "This project showed me the full path of an analytics problem: understand the data, find the pattern, build a simple model, and question my own result. In a fraud team I would use the same thinking: look for what makes risky transactions different, and measure both the fraud we catch and the good customers we disturb."

9. Tips for speaking clearly

A simple structure for any answer

Headline, then reason, then example. Example: "I used Logistic Regression (headline) because it is simple and gives a probability (reason). It reached about 92 percent on test data (example)."

Pace and voice

- Speak slower than you think you need to. Breathe at the end of every sentence.
- Keep sentences short. One idea per sentence.
- Say numbers clearly and slowly: "two hundred eighty-four thousand, eight hundred seven transactions". Or just say "about 285,000".
- It is fine to pause for 2 or 3 seconds to think. Interviewers like calm people.

If you forget something

- Say: *"Let me think for a second."* Then breathe and go back to the headline of the answer.
- Use the memory trick: Problem, Data, Fix, Model, Result, Learning. Find where you stopped.
- If you do not know: *"I am not fully sure about that. My understanding is ..., and I would check it."* Honest is better than a wrong guess.
- If they ask about something you did not do: *"I did not do that in this project, but here is how I would do it."*

Before the interview

- Read the 30-second and 2-minute answers aloud 3 times. Stand up if you can.
- Remember these 6 numbers: 284,807 rows, 492 fraud, 0.17 percent, 984 balanced rows, 93.5 percent train, 91.9 percent test.
- Open your notebook once and be ready to show it. Know what each cell does.
- Sleep well. A calm mind beats extra revision.

10. One-page cheat sheet

The story in 6 words

Problem - Data - Fix - Model - Result - Learning

Key numbers

- 284,807 transactions, 31 columns
- 492 fraud = 0.17%
- Balanced sample: 492 + 492 = 984
- Split 80/20: 787 train, 197 test
- Train accuracy 93.5%, test accuracy 91.9%

The tools

Python, pandas, numpy, scikit-learn. Model: Logistic Regression.

One-line meanings

- **Imbalanced:** fraud is very rare.
- **Under-sampling:** use fewer normal rows to match fraud.
- **Stratify:** same fraud share in train and test.
- **Overfitting:** great on practice, weak on exam.
- **Precision:** when I say fraud, am I right?
- **Recall:** how much real fraud did I catch?
- **PCA:** mixed hidden columns for privacy.

Be honest about

- Only accuracy was measured.
- Test set was balanced, not real life.
- Notebook has one model (Logistic Regression).
- README claims of 100% for KNN and DT: only mention if you can prove them.
- Normal sample had no fixed seed.

My next steps

- Confusion matrix, precision, recall, F1
- Test on real fraud rate
- SMOTE or class weights
- Random Forest and XGBoost
- Scale Time and Amount; use cross-validation

Last-minute reminders

- Slow down. Short sentences.
- Headline first, then reason, then example.
- Agree honestly with weak points, then say how you would fix them.
- Never claim a result you cannot show. You are good enough with the truth.